Reviewer Pack — Minimal Alexander-Orlik-Solomon Complexity and Resolution Pr…

Entry f303128ed344 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 00:54:00 UTC

Minimal Alexander-Orlik-Solomon Complexity and Resolution Proof Width

Entry ID: `f303128ed344` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 00:54:00 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Alexander Theory **Field B** (complexity object): Boolean Satisfiability (Resolution Proof Complexity)

Statement:

> The minimal Alexander-Orlik-Solomon complexity of a d-regular graph G, defined as the smallest number of generators required for the Alexander module of G in the Orlik-Solomon algebra, is linearly correlated with its resolution proof width w(φ_G), such that the minimal Alexander-Orlik-Solomon complexity AOS(G) = Θ(w(φ_G)).

Rationale (proposer's reasoning):

> The Alexander-Orlik-Solomon complexity captures the topological properties of a graph through its Alexander module, which may reveal hidden structures that are not immediately apparent from the graph's connectivity. This correlation could potentially expose new insights into the resolution proof process.

Taxonomy category: `Alexander-Orlik-Solomon_Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `91674378e79b8999`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 80% of the graphs have an Alexander-Orlik-Solomon complexity within a factor of 3 of their resolution proof width, and no graph has a discrepancy greater than 10.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Alexander-Orlik-Solomon complexity" AND "resolution proof width" - "combinatorial Alexander theory" AND "Boolean satisfiability" - "d-regular graph" AND "Alexander module" AND "resolution proof complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if (n * d) % 2 != 0 or d >= n:
            return None
        graph = [[] for _ in range(n)]
        edges_used = set()
        for i in range(d):
            for j in range(i + 1, n):
                if (i, j) not in edges_used and (j, i) not in edges_used:
                    graph[i].append(j)
                    graph[j].append(i)
                    edges_used.add((i, j))
        return graph

    def tseitin_formula(graph, n):
        clauses = []
        literals = {}
        for v in range(n):
            literals[v] = random.randint(1, 2 * n)
        for v in range(n):
            clauses.append([literals[v]])
            for u in graph[v]:
                clauses.append([-literals[v], literals[u]])
        return clauses

    def resolution_width(clauses):
        queue = clauses[:]
        learned_clauses = []
```

```
    while queue:
        clause1 = queue.pop()
        if len(clause1) == 0:
            return float('inf')
        for clause2 in queue + learned_clauses:
            if len(clause2) == 0:
                return float('inf')
            for literal in clause1:
                if -literal in clause2:
                    new_clause = [l for l in clause2 if l != -literal]
                    if new_clause not in queue and new_clause not in learned_clauses:
                        learned_clauses.append(new_clause)
                        break
    return max(len(clause) for clause in learned_clauses)

def alexander_orlik_solomon_complexity(graph):
    n = len(graph)
    generators = set()
    for v in range(n):
        for u in graph[v]:
            if v < u:
                generators.add((v, u))
    return len(generators)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    d = random.randint(1, min(n - 1, 2))
    graph = generate_d_regular_graph(n, d)
    if graph is None:
        continue
    clauses = tseitin_formula(graph, n)
    width = resolution_width(clauses)
    complexity = alexander_orlik_solomon_complexity(graph)
    results.append({
        "n": n,
        "d": d,
        "complexity": complexity,
        "width": width
    })

if not results:
    return {
        "metric_name": "resolution_width",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

total_complexity = sum(result["complexity"] for result in results)
total_width = sum(result["width"] for result in results)
mean_complexity = total_complexity / len(results)
mean_width = total_width / len(results)
```

```
    if any(abs(result["complexity"] - result["width"]) > 10 for result in results):
        return {
            "metric_name": "resolution_width",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": max(result["n"] for result in results),
            "conjecture_holds": False,
            "counterexample": "discrepancy_greater_than_10"
        }

    if any(abs(mean_complexity - mean_width) > 3 * (mean_width / len(results)) for _ in range(10)):
        return {
            "metric_name": "resolution_width",
            "metric_value": None,
            "instances_tested": len(results),
            "n_max": max(result["n"] for result in results),
            "conjecture_holds": False,
            "counterexample": "correlation_factor_greater_than_3"
        }

    return {
        "metric_name": "resolution_width",
        "metric_value": mean_complexity,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results if result["metric_value"] is not
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results if
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(result["counterexample"] == "discrepancy_greater_than_10" for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["countere
        print(f"RESULT: FALSIFIED counterexample=\"discrepancy_greater_than_10\" first_failing_see
    elif any(result["counterexample"] == "correlation_factor_greater_than_3" for result in results
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["countere
        print(f"RESULT: FALSIFIED counterexample=\"correlation_factor_greater_than_3\" first_faili
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
on_width', 'metric_value': None, 'instances_tested': 5, 'n_max': 40, 'conjecture_holds': False, 'cour
TRIAL: {'metric_name': 'resolution_width', 'metric_value': None, 'instances_tested': 4, 'n_max': 40,
TRIAL: {'metric_name': 'resolution_width', 'metric_value': None, 'instances_tested': 4, 'n_max': 40,
TRIAL: {'metric_name': 'resolution_width', 'metric_value': None, 'instances_tested': 6, 'n_max': 40,
TRIAL: {'metric_name': 'resolution_width', 'metric_value': None, 'instances_tested': 5, 'n_max': 40,
TRIAL: {'metric_name': 'resolution_width', 'metric_value': None, 'instances_tested': 5, 'n_max': 40,
TRIAL: {'metric_name': 'resolution_width', 'metric_value': None, 'instances_tested': 4, 'n_max': 40,
TRIAL: {'metric_name': 'resolution_width', 'metric_value': None, 'instances_tested': 5, 'n_max': 40,
TRIAL: {'metric_name': 'resolution_width', 'metric_value': None, 'instances_tested': 5, 'n_max': 40,
TRIAL: {'metric_name': 'resolution_width', 'metric_value': None, 'instances_tested': 4, 'n_max': 40,
TRIAL: {'metric_name': 'resolution_width', 'metric_value': None, 'instances_tested': 5, 'n_max': 40,
RESULT: FALSIFIED counterexample="discrepancy_greater_than_10" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test code only tests up to n = 40, which is too small to confidently assess the conjecture's validity. The metric does not scale trivially with n, but a larger sample size is needed to confirm or refute the conjecture.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results indicate that for at least one seed (first_failing_seed=11), the discrepancy between the Alexander-Orlik-Solomon complexity and the r | next: Increase the size of the graph instances tested to a larger scale, ensuring that the sample size is sufficient to accurately assess the conjecture. Additionally, consider implementing a more robust testing framework that can handle larger graphs and provide more reliable results.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14196
2	propose	ollama_remote	glm4:latest	0	0	12506
3	preregistration	ollama_remote	glm4:latest	0	0	9557
4	novelty	ollama_remote	glm4:latest	0	0	8594
5	novelty	ollama_remote	glm4:latest	0	0	8977
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44679
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13820
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17329
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16184
10	critic	ollama_remote	glm4:latest	0	0	16478

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
11	judge	ollama_remote	glm4:latest	0	0	9403

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 171723 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/f303128ed344.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f303128ed344.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f303128ed344.tar.gz` (if generated)